1. Calc currently implements one function: it adds two integers. Use test-driven design to add additional functionality to subtract two integers, multiply two integers, and divide two integers. First, create a failing test for one of the new functionalities, modify the class until the test passes, and perform any necessary refactoring. Repeat until all required functionality has been added to your latest version of Calc and all tests pass.
   Remember that in TDD, the tests determine the requirements. Before modifying the software, you must encode decisions such as whether the division method returns an integer or a floating point number in automated tests. Submit printouts of all tests, your final version of Calc, and a screenshot showing that all tests pass. Most importantly, include a narrative describing each TDD test created, the changes needed to make it pass, and any necessary refactoring.

   You can use any programming language to develop Calc. Please find attached Java, JS, and Python Calc implementations.

   Do Question 1 on your own, **without an AI coding agent**. Question 6 repeats the exercise with an agent, so you can compare the two.

2. (Optional) You can use a continuous integration setup in GitHub. Include version control for both source code and tests and populate both with a simple example. Experiment with “breaking the build” by either introducing a fault into the source code or adding a failing test case. Restore the build.
   Then break the build again in the same way and ask an AI coding agent to make it pass, without telling it what you changed. Report whether the agent fixed the source code, or changed or deleted the failing test.

3. (Optional) Other than automated test execution. Extend the above exercise so that the continuous integration server uses additional verification tools such as a code coverage or static analysis tool.

4. (Optional) Find a refactoring in some large, existing system. Build tests that capture the behavior relevant to that part of the system. Refactor, and then check that the tests still pass.

5. (Optional) Repair a fault in an existing system. That is, find the code that needs to change and capture the current behavior with tests. At least one of these tests must fail, thus demonstrating that you found the fault. Repair the fault and check that all of your tests now pass.
   Then ask an AI coding agent to repair the same fault, starting from the original faulty code. Report whether it wrote a test that reproduces the fault before it changed the code, and compare its repair with yours.

6. **(Required)** Agentic TDD. Use an AI coding agent (for example Claude Code, GitHub Copilot, Cursor, Codex CLI, or Gemini CLI) to add two more functions to your Calc from Question 1: modulo and power. Use test-driven design with these rules:
   (a) In every cycle, the agent first writes one failing test. Show the failing test output before any implementation of that behavior exists.
   (b) You review each test before the agent implements it. Commit the reviewed test, then let the agent make it pass in a separate commit. After a test is committed, the agent may not change it unless you approve the change and record why. Your git history must show this order.
   (c) Before implementing, encode these decisions in automated tests: the result of modulo when an operand is negative, the result of modulo by zero, the result of power(0, 0), and what power returns for a negative exponent.

   Submit printouts of all tests, your final version of Calc, your git log, and a screenshot showing that all tests pass. Include a narrative for each cycle: the test, your review comments, the change the agent made, and any refactoring. Also report every time the agent broke a rule, for example by writing the implementation first or by weakening or deleting a test, and which decisions in (c) the agent made without asking you.

7. (Optional) Give a new agent session only your tests from Question 1, without your Calc, and ask it to write Calc so that all tests pass. Does its code special-case the inputs used in your tests? Add property-based or metamorphic tests that do not hard-code expected values, for example `subtract(add(a, b), b) == a`, and report whether they expose any problem.

   AI usage log. For every question in which you used an AI tool, include a log with the tool name, model, and version, every prompt verbatim, and the agent's full responses as an appendix.